PlanToBee

Uygulama ve Backend Mimarisi Raporu

Ailelerin haftalık ders, aktivite ve okul planını tek ekranda paylaştığı platformun teknik yapısı; Render, Neon ve Brevo servislerinin seçilme gerekçeleri.

Tarih: 5 Kasım 2026
Kaynak: iisler/PlanToBee deposu (backend/PlanToBee.API, render.yaml, docs/DEPLOY.md)

İçindekiler

1. Yönetici Özeti 3
2. PlanToBee Uygulaması 3
2.1 Amaç ve kullanıcılar 3
2.2 Temel özellikler 3
3. Genel Mimari 4
4. Backend Mimarisi 5
4.1 Teknoloji yığını 5
4.2 Katmanlar ve klasör yapısı 5
4.3 API uç noktaları 6
4.4 Veri modeli 6
4.5 Kimlik doğrulama ve oturum 7
4.6 Güvenlik ve gizlilik 7
4.7 Bildirim altyapısı (Web Push) 8
4.8 Açılış akışı 8
5. Neden Render? 8
6. Neden Neon (neon.tech)? 9
7. Neden Brevo? 10
8. Servislerin Karşılaştırmalı Özeti 11
9. Dağıtım, Test ve Yedekleme 11
10. Yol Haritası: Aşama 2 12

1. Yönetici Özeti

PlanToBee, bir ailenin ortak haftalık planını (dersler, spor ve diğer aktiviteler, okul programı) tek ekranda tuttuğu, telefonda uygulama gibi çalışan bir web uygulamasıdır (PWA). Aile tek bir hesapla giriş yapar; cihazda "Kim kullanıyor?" ekranından profil seçilir. Ebeveyn ve çocuk profilleri aynı planı görür, ama yetkileri farklıdır.

Sistem dört parçadan oluşur ve hepsi bugün **ücretsiz planlarda** çalışır:

Parça	Servis	Görevi
Frontend	GitHub Pages	React 19 + Vite ile derlenen statik site. Kullanıcının tarayıcısına iner.
Backend (API)	**Render** (Docker web servisi)	.NET 10 / ASP.NET Core REST API. İş kuralları, kimlik doğrulama, yetki, bildirim.
Veritabanı	**Neon** (sunucusuz PostgreSQL)	Kullanıcılar, aileler, profiller, plan kayıtları, bildirim kuyruğu.
E-posta	**Brevo** (SMTP relay)	E-posta doğrulama, şifre sıfırlama ve girişte 6 haneli kod e-postaları.

> **Kısaca neden bu üçü?**
>
> **Render:** Kodu Docker imajı olarak, depodaki `render.yaml` ile tek tıkla ve `main` dalına her push'ta otomatik yayınlar; HTTPS'i ve gizli anahtar üretimini kendisi yapar.
>
> **Neon:** Yönetilen, boştayken uyuyan ve bu sayede ücretsiz kalan gerçek bir PostgreSQL. Render ile aynı bölgede (Frankfurt) çalışır.
>
> **Brevo:** Render'ın ücretsiz planı giden e-posta portlarını (25, 465, 587) engellediği için Gmail doğrudan kullanılamaz. Brevo açık kalan **2525** portundan şifreli (STARTTLS) gönderim yapar; günde 300 e-posta ücretsizdir.

2. PlanToBee Uygulaması

2.1 Amaç ve kullanıcılar

Uygulama ilk olarak bir lise öğrencisinin ders çalışma planını ve voleybol antrenmanlarını takip etmek için tek dosyalık bir Firebase sayfası olarak başladı. Yeni sürüm (React + .NET) bunu çok kullanıcılı, güvenli bir **aile planlayıcısına** dönüştürdü:

- **Ebeveyn:** Planı kurar, profilleri yönetir, ilerlemeyi takip eder. Profili 4 haneli PIN ile korunur.
- **Çocuk / öğrenci:** Dersini ve aktivitelerini ekler, tamamlananı işaretler. Başkasının kaydını düzenleyemez.
- **Hesap sahibi (admin):** Aileyi kuran ebeveyn; silinemez, ailenin e-posta ve şifresinin sahibidir.

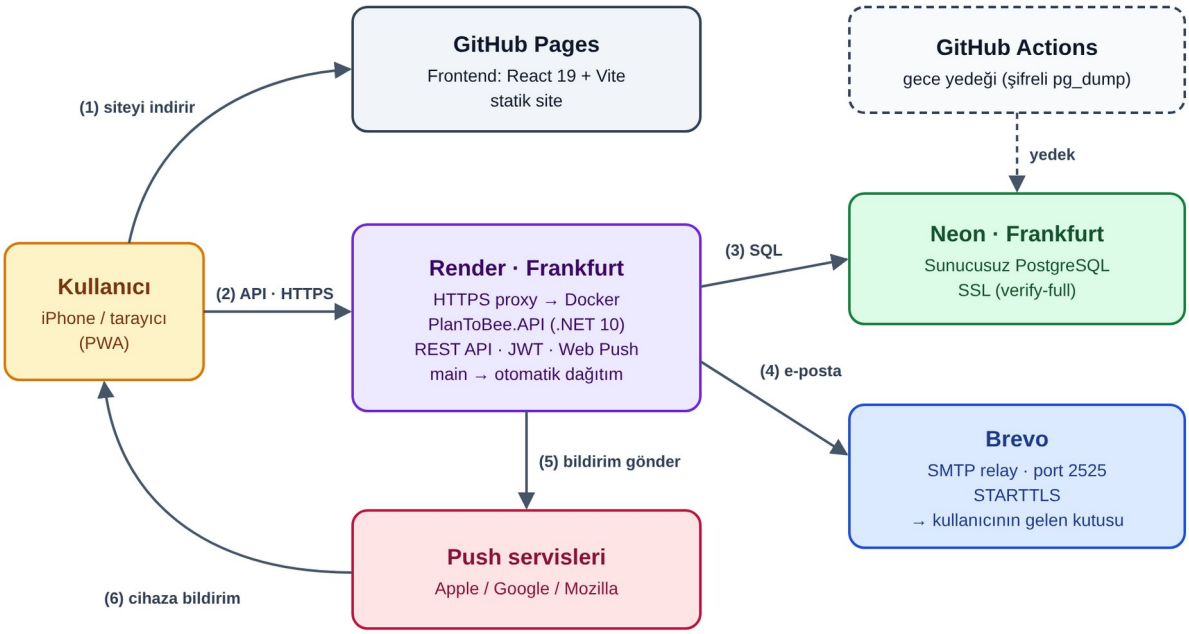
2.2 Temel özellikler

Özellik	Açıklama
Aile hesabı ve profiller	Netflix tarzı profil seçimi. Ebeveyn profilleri PIN'li, çocuk profilleri isteğe bağlı PIN'li. 5 hatalı PIN'de profil 5 dakika kilitlenir.

Özellik	Açıklama
Ortak plan	Tüm aile aynı günleri ve kayıtları görür; her kayıtta "ekleyen / düzenleyen" izi tutulur.
Dersler (To-do)	Ders + konu + süre (+ isteğe bağlı başlangıç saati). Durum: Yapılacak → Devam ediyor → Tamamlandı. Ailenin ortak ders listesi.
Aktiviteler	Spor, Müzik, Konser, Buluşma, Sınav, Okul, Diğer. Başlangıç–bitiş saati, not.
Tekrarlayan aktiviteler	Haftalık kural (ör. "Haziran'a kadar her hafta içi okul") tek satır olarak saklanır; günlere okunurken yayılır. Tek günü atlama, geri alma, seriden ayırma.
Saat çizelgesi	Gün için dikey zaman çizelgesi, hafta için 7 sütunlu zaman ızgarası.
Program yükleme	Excel, CSV ya da fotoğraftan (cihazda OCR, Tesseract) haftalık programı toplu içe aktarma; tek tuşla "Geri al".
Sesle ekleme	Aktiviteyi konuşarak hızlı ekleme.
Bildirimler	Web Push: aile üyesi kayıt eklediğinde, değiştirdiğinde ya da ders bitirdiğinde bildirim. Sessiz saatler ve kişi bazlı susturma.
Resmî tatiller	Türkiye resmî tatilleri ve MEB okul tatilleri; tatilde "Okul" tekrarları gösterilmez.
KVKK	Aydınlatma metni, kullanım koşulları ve uygulama içinden hesap silme.

3. Genel Mimari

Aşağıdaki şema, bir kullanıcının uygulamayı açmasından bildirim almasına kadar parçaların nasıl konuştuğunu gösterir.



Şekil 1. PlanToBee bileşenleri ve veri akışı

1. Tarayıcı, statik React uygulamasını **GitHub Pages**'ten indirir. Uygulama telefonda ana ekrana eklenebilir (PWA, service worker).

2. Uygulama, API adresini derleme sırasında `VITE_API_URL` değişkeninden öğrenir ve tüm istekleri HTTPS ile **Render**'a gönderir. Render'ın proxy'si HTTPS'i çözer ve isteği Docker konteynerindeki .NET API'ye iletir.
3. API, **Neon** PostgreSQL'e sertifikası doğrulanan SSL (`verify-full`) ile bağlanır.
4. Doğrulama, şifre sıfırlama ve giriş kodu e-postaları **Brevo** SMTP üzerinden (port 2525, STARTTLS) kullanıcının gelen kutusuna gider.
5. Bildirimler, API'nin kendi yazdığı Web Push şifrelemesiyle Apple / Google / Mozilla push servislerine, oradan cihaza gider.
6. GitHub Actions her gece veritabanının şifreli yedeğini alır.

API yalnızca `App__FrontendBaseUrl` ayarındaki sitenin alan adından gelen tarayıcı isteklerine izin verir (CORS). Aynı ayar e-postalardaki bağlantıların adresini de belirler; alan adı değiştiğinde tek ayar güncellenir.

4. Backend Mimarisi

4.1 Teknoloji yığını

Bileşen	Seçim
Çalışma zamanı	.NET 10, ASP.NET Core (Web API, controller tabanlı)
Veri erişimi	Entity Framework Core 10 + Npgsql (PostgreSQL sağlayıcısı); code-first migration'lar
Kimlik	ASP.NET Core Identity (şifre özeti, kilitleme, belirteç sağlayıcıları) + JWT Bearer
Anahtar yönetimi	ASP.NET Core Data Protection; anahtarlar veritabanında, AES-256-GCM ile şifreli
Paketleme	Çok aşamalı Dockerfile (SDK ile derleme, yalnızca aspnet çalışma zamanı imajı, root olmayan kullanıcı)
Dış bağımlılık	Yalnızca 5 NuGet paketi (hepsi Microsoft / Npgsql). Web Push, e-posta, oran sınırlama için ek kütüphane yok.
Test	`backend/tests/e2e.py` (~1.500 satır uçtan uca API senaryosu) + Web Push şifreleme öz testi; her backend PR'ında GitHub Actions'ta geçici PostgreSQL ile çalışır.

4.2 Katmanlar ve klasör yapısı

Klasör	Sorumluluk
Controllers/	HTTP uç noktaları: Auth, Account, Family, Profiles, Days, Series, Subjects, Push. İnce tutulur; işi servislere devreder.
DTOs/	İstek/yanıt sözleşmeleri (Auth, Family, Day, Series). Veritabanı modeli dışarıya doğrudan açılmaz.
Services/	İş mantığı: `MemberContext` (istek sahibinin hesabı, ailesi, seçili profili), `FamilyService`, `AuthTokenService`, `PinService`, `PlanWrites`, `SeriesSchedule` (tekrarları tarihe yayma), `Holidays`, bildirimler, e-posta.

Klasör	Sorumluluk
Models/	Varlıklar: User, Family, FamilyMember, Day, StudyEntry, Event, EventSeries, Subject, RefreshToken, Push.
Data/	`AppDbContext` (EF Core), değer dönüştürücüler (kişisel veri şifreleme), Türkçe Identity hata mesajları.
Infrastructure/	Kesişen konular: açılış ayar doğrulaması, bağlantı dizesi, oran sınırları, proxy başlıkları, sağlık kontrolleri, şifreleme, Web Push kriptografisi, yönetici komutları, Firebase aktarımı.
Migrations/	15 EF Core migration'ı; API açılışta bekleyenleri kendisi uygular.

4.3 API uç noktaları

Tüm uç noktalar `/api` altında, JSON ile çalışır. Hatalar her zaman `{ code, message }` biçimindedir; yığın izi ya da SQL istemciye gönderilmez.

Grup	Başlıca uç noktalar
/api/auth	register, login, login/verify (e-posta kodu), login/resend-code, refresh, logout, me, verify-email, resend-verification, forgot-password, reset-password
/api/family	Aileyi getir, oluştur, adını güncelle
/api/profiles	Profilleri listele, seç (PIN), ekle, düzenle, PIN değiştir, sil
/api/days	Gün ve hafta görünümü, hafta ayrıntısı; ders (entries) ekle/düzenle/durum değiştir/sil; aktivite (events) ekle/düzenle/sil
/api/series	Tekrarlayan aktivite: oluştur, düzenle, sil, tek günü atla/geri al, seriden ayır; toplu yükleme (bulk) ve toplu geri alma
/api/subjects	Ailenin ders listesi: listele, ekle, sil
/api/push	Bildirim yapılandırması, abonelik ekle/sil, ayarlar, cihaz sil, test bildirimi
/api/account	Hesap silme (KVKK)
/health, /health/live	Sağlık kontrolü: `/health` veritabanını da yoklar; `/health/live` yalnızca API'yi (Render bunu kullanır)

4.4 Veri modeli

Plan ailenin ortak planıdır: her şey `FamilyId` üzerinden aileye bağlanır. Profil (FamilyMember) ise kayıtların izinde ve yetki kontrolünde kullanılır.

Tablo	İçerik
Users	Identity hesabı. E-posta ve kullanıcı adı **şifreli** saklanır.
Families	Aile: adı, oluşturulma zamanı.
FamilyMembers	Profil: ad, rol (Parent/Child), durum (Active/Left), IsAdmin, PIN özeti (PBKDF2), hatalı deneme sayacı.
Days	Ailenin bir günü (FamilyId + tarih).
StudyEntries	Ders kaydı: ders, konu, dakika, başlangıç saati, durum.

Tablo	İçerik
Events	Aktivite: tür (metin olarak, yeni tür migration gerektirmez), başlık, saat, bitiş, not.
EventSeries (+ Exceptions)	Haftalık tekrar kuralı: günler bit maskesi, başlangıç–bitiş tarihi (en çok 1 yıl), istisna günleri, toplu yükleme kimliği.
Subjects	Ailenin ders listesi.
RefreshTokens	Oturum yenileme belirteçlerinin yalnızca SHA-256 özeti.
PushSubscriptions, NotificationPreferences, PendingNotifications	Cihaz abonelikleri (şifreli), profil bildirim ayarları, gönderilmeyi bekleyen bildirim kuyruğu.
DataProtectionKeys	Doğrulama/sıfırlama bağlantılarını imzalayan anahtarlar (şifreli).

Kayıtların ortak atası `AuditedEntity`, her kayda ekleyen/düzenleyen profili ve zamanı ekler. Profil silinse bile satır "Eski üye" olarak tutulur, böylece geçmiş kayıtların izi kaybolmaz.

4.5 Kimlik doğrulama ve oturum

- **İki belirteçli oturum:** 15 dakikalık JWT erişim belirteci + 30 günlük yenileme belirteci. Yenileme belirteci her kullanımda değişir (rotation); kullanılmış bir belirteç tekrar gelirse çalınmış sayılır ve kullanıcının tüm oturumları kapatılır.
- **Security stamp kontrolü:** Her istekte belirteçteki damga veritabanıyla karşılaştırılır; şifre değişince eski oturumlar anında geçersiz olur.
- **E-posta ile iki adımlı giriş:** Doğrulanmış hesaplarda girişte e-postaya 6 haneli kod gider (10 dk geçerli, tek kullanımlık, 5 hatada iptal). Kodlar Brevo üzerinden gönderilir.
- **Profil PIN'i:** Aile şifresini bilen biri ebeveyn profilini seçemesin diye ebeveyn profilleri PIN zorunludur. PIN'ler PBKDF2 ile özetlenir.
- **E-posta doğrulaması:** Doğrulanmamış hesap giriş yapabilir ama aile/plan uç noktaları `[RequireVerifiedEmail]` ile kapalıdır.

4.6 Güvenlik ve gizlilik

Önlem	Nasıl uygulanıyor
Kişisel veri şifreleme	E-posta adresleri AES-256-GCM ile şifreli; aranan alanlar HMAC-SHA256 ile anahtarlı özet. Veritabanı ya da yedeği sızsa bile adresler okunamaz. Anahtar yalnızca Render ortam değişkeninde (`PersonalData__Key`).
Anahtar şifreleme	Data Protection anahtarları veritabanına AES-256-GCM ile şifreli yazılır; şifreleme anahtarı ortam değişkenindedir.
Oran sınırlama	Giriş/kayıt: IP başına dakikada 20. PIN: aile başına 5 dakikada 20. Plan uç noktaları: kullanıcı başına dakikada 300. Program yükleme: dakikada 10.
Proxy arkasında gerçek IP	`X-Forwarded-For` yalnızca en sağdaki (Render'ın eklediği) değerle okunur (ForwardLimit=1); sahte başlıkla sınır atlatılamaz. IPv6 /64 bloklarına göre gruplanır.
Taşıma güvenliği	Render HTTPS sonlandırır, API HSTS başlığı ekler. Veritabanı bağlantısı `verify-full` SSL'e yükseltilir; `sslmode=disable` ile API açılmaz.
CORS	Yalnızca sitenin kökenine (ör. `https://iisler.github.io`) izin verilir.

Önlem	Nasıl uygulanıyor
Açılış doğrulaması	Üretimde eksik/zayıf ayar varsa (ör. kısa JWT anahtarı, SMTP bilgisi yok, depoda sızmış eski anahtar) API anlaşılır bir mesajla açılmayı reddeder.
Eşzamanlılık	Kayıt aynı anda başka bir cihazdan değiştirilmiş ya da silinmişse istek 409 ile döner; ders listesine yazımda PostgreSQL danışma kilidi (advisory lock) kullanılır.
Loglar	Şifre, bağlantı dizesi, e-posta gövdesi, kod ya da bağlantı loglanmaz.

4.7 Bildirim altyapısı (Web Push)

Bildirimler dış servis ya da kütüphane kullanılmadan, standartlara (RFC 8291 şifreleme, RFC 8292 VAPID) uygun şekilde API içinde yazıldı. VAPID anahtarı tek bir gizli değerden (`WebPush__Key`) türetilir.

- Kayıt eklendiğinde olay `PendingNotifications` kuyruğuna yazılır ve arka plan servisi (`NotificationDispatcher`) bir sinyalle uyanır.
- Aynı kişinin 60 saniye içindeki ardışık eklemeleri tek bildirimde toplanır ("Ela 3 kayıt daha ekledi").
- Sessiz saatler (varsayılan 22:00–07:30) ve kişi bazlı susturma desteklenir; vakti gelen bildirim sessiz saat bitince gider.
- **Önemli tasarım kararı:** Dağıtıcı kuyruğu düzenli aralıkla sorgulamaz; yalnızca yeni olayda ya da bekleyen bir bildirimin vakti gelince uyanır. Böylece trafik yokken veritabanına hiç gidilmez ve Neon uyuyabilir (bkz. Bölüm 6).

4.8 Açılış akışı

1. Ayar doğrulaması (`StartupValidation`): zorunlu gizli değerler, bağlantı dizesi, SMTP, URL biçimleri.
2. Servis kayıtları: veritabanı, Identity, JWT, Data Protection, oran sınırları, bildirim, e-posta göndericisi.
3. Bekleyen migration'lar uygulanır (boş veritabanında tüm tablolar oluşur), şifrelenmemiş eski e-postalar şifrelenir.
4. Ara katmanlar: proxy başlıkları → HSTS → hata yakalayıcı → sağlık kontrolleri → CORS → kimlik doğrulama → oran sınırı → yetkilendirme → controller'lar.
5. API, Render'ın verdiği `PORT` üzerinden dinlemeye başlar.

5. Neden Render?

Render, backend'i (PlanToBee.API) barındıran platformdur. Docker imajını derler, çalıştırır, önüne HTTPS proxy koyar ve kodu her `main` güncellemesinde otomatik yayınlar.

Bize sağladıkları

- **Altyapı kod olarak (Blueprint):** Servisin tüm ayarları depodaki `render.yaml` dosyasındadır: Docker çalışma zamanı, Frankfurt bölgesi, ücretsiz plan, `main` dalı, sağlık kontrolü yolu ve ortam değişkenleri. Kurulum "New → Blueprint → depoyu seç" kadar basittir ve tekrarlanabilir.
- **Gizli anahtarları kendisi üretir:** `Jwt__Key`, `PersonalData__Key`, `DataProtection__KeyEncryptionKey` ve `WebPush__Key` değerlerini Render rastgele üretir (`generateValue: true`). Gizliler hiçbir zaman depoya girmez; yalnızca Neon adresi ve Brevo bilgileri kurulumda elle girilir (`sync: false`).

- **Docker desteği:** .NET 10 için özel bir çalışma zamanı beklemeden, kendi Dockerfile'ımızla çalışır. Aynı imaj yerelde ve başka bir sağlayıcıda da çalışır (sağlayıcıya kilitlenme yok).
- **Otomatik dağıtım ve sağlık kontrolü:** `main`'e gelen her değişiklik yeniden derlenir; Render yeni sürüm `/health/live`'a sağlıklı cevap verene kadar eski sürümü yayında tutar. Açılamayan sürüm yayına girmez.
- **Yönetilen HTTPS:** TLS sertifikası ve HTTP→HTTPS yönlendirmesi Render'dadır; uygulama içinde sertifika yönetimi gerekmez. Özel alan adı (`api.plantobee.com`) da aynı ekrandan eklenir.
- **Frankfurt bölgesi:** Türkiye'deki kullanıcılara yakın ve Neon ile aynı bölge; API–veritabanı gecikmesi düşük kalır.
- **Ücretsiz plan:** Bu aşamadaki trafik için yeterlidir.

Kısıtlar ve aldığımız önlemler

Kısıt	Önlem
15 dakika istek gelmezse servis uyur; ilk istek 30–60 sn sürebilir.	Kabul edildi (aile uygulaması, düşük trafik). Uyandırınca konteyner sıfırdan başlar; bu yüzden imza anahtarları veritabanında saklanır ve e-posta bağlantıları uykudan sonra da geçerli kalır.
Ücretsiz planda giden e-posta portları (25, 465, 587) kapalı.	Brevo'nun açık olan 2525 portu kullanılır (Bölüm 7).
Konteyner diski kalıcı değil.	Tüm durum Neon'da; bellekte tutulan yalnızca oran sınırı sayaçları ve bekleyen giriş kodlarıdır (yeniden başlatmada sıfırlanması kabul edilebilir).
Proxy IP aralığı yayınlanmıyor.	ForwardLimit=1 ile yalnızca Render'ın eklediği IP'ye güvenilir.

6. Neden Neon (neon.tech)?

Neon, PostgreSQL'i sunucusuz (serverless) olarak sunan yönetilen bir veritabanı hizmetidir. Depolama ile işlem gücünü ayırır: veritabanı boştayken işlem birimi uyur, ilk sorguda saniyenin altında uyanır.

Bize sağladıkları

- **Gerçek PostgreSQL:** Kod tarafında hiçbir özel uyarlama yok; standart Npgsql + EF Core ile çalışır. İleride başka bir PostgreSQL sağlayıcısına geçmek yalnızca bağlantı adresini değiştirmektir.
- **Uygulamanın ihtiyaç duyduğu PostgreSQL özellikleri:** Danışma kilitleri (`pg_advisory_xact_lock`), kısmi benzersiz indeksler (ailede tek admin), işlemsel migration'lar.
- **Ücretsiz plan ve uyuma:** Trafik yokken 5 dakika sonra uyur; ücretsiz planın aylık işlem süresi bu sayede yeter. Backend bunun için özellikle tasarlandı: Render'ın sağlık kontrolü veritabanına dokunmayan `/health/live`'ı kullanır, bildirim dağıtıcısı da kuyruğu boşuna sorgulamaz.
- **Yönetim yükü yok:** Kurulum, güncelleme, yedekleme altyapısı, SSL sertifikası Neon'dadır. Bağlantı `verify-full` ile, sertifika ve sunucu adı doğrulanarak kurulur.
- **Dallar (branch):** Şema değiştiren bir sürümden önce veritabanının o anki kopyası saniyeler içinde "branch" olarak alınabilir; geri dönüş planlarımız bunu kullanır.
- **Bölge uyumu:** AWS Europe Central 1 (Frankfurt) — Render ile aynı bölge.
- **Bağlantı adresi doğrudan kullanılır:** API, Neon'un verdiği `postgresql://...` adresini olduğu gibi kabul eder.

Kısıtlar ve aldığımız önlemler

Kısıt	Önlem
Ücretsiz planda geçmişe dönüş penceresi kısa (saatler).	Her gece GitHub Actions ile `pg_dump` alınır, AES-256 ile şifrelenir, 30 gün saklanır. Geri yükleme de ayrı bir iş akışıyla, boş bir veritabanına tek işlem olarak yapılır.
Aylık işlem süresi sınırlı.	`/health/live` ve olay tabanlı bildirim dağıtıcısı sayesinde boşta veritabanına gidilmez.
Havuzlu (pooler) bağlantı migration'larda güvenilir değil.	Doğrudan bağlantı kullanılır; havuzlu adres girilirse API uyarı yazar.

7. Neden Brevo?

Brevo (eski adıyla Sendinblue), işlemsel (transactional) e-posta gönderen bir servistir. PlanToBee'de üç tür e-posta gönderir: **e-posta adresi doğrulama**, **şifre sıfırlama** ve **girişte 6 haneli doğrulama kodu**.

> **Asıl neden: Render portları**
>
> Render'ın ücretsiz planı Eylül 2025'ten beri giden SMTP portlarını (**25, 465, 587**) engelliyor. Gmail SMTP yalnızca bu portlarla çalıştığı için Render'dan doğrudan Gmail ile e-posta gönderilemez.
>
> Brevo ise **2525** portunu da destekler ve bu port Render'da açıktır. API bağlantıyı düz açar, `STARTTLS` ile şifreler; kullanıcı adı ve anahtar yalnızca şifreli bağlantı kurulduktan sonra gönderilir.

Diğer kazanımlar

- **Standart SMTP:** Kod, .NET'in yerleşik SMTP istemcisini kullanır; Brevo'ya özel kütüphane yoktur. Sağlayıcı değiştirmek yalnızca ortam değişkenlerini değiştirmektir.
- **Ücretsiz kota:** Günde 300 e-posta; bir aile uygulaması için fazlasıyla yeterli.
- **Teslim edilebilirlik:** Gönderen adres doğrulaması, ileride kendi alan adımız (`plantobee.com`) için SPF/DKIM kayıtları; e-postaların spam'e düşme olasılığı azalır.
- **İzlenebilirlik:** Brevo panelinde (Transactional → Logs) her e-postanın ulaşıp ulaşmadığı görülür; sorun gidermede API logları ile birlikte kullanılır.
- **Kişisel hesaptan ayrım:** Gönderim bu iş için açılmış ayrı bir hesaptan yapılır; kişisel Gmail hesabının şifresi ya da uygulama şifresi hiçbir yere girilmez.

Kısıtlar ve aldığımız önlemler

Kısıt	Önlem
Gmail adresi gönderen olarak kullanılınca e-postalar spam'e düşebilir.	Aşama 2'de `noreply@plantobee.com` alan adı Brevo'da SPF/DKIM ile doğrulanacak.
Girişte e-posta kodu açıkken Brevo kesilirse kimse giriş yapamaz.	Acil durumda `Auth__EmailCode__Enabled=false` ile kod adımı kapatılabilir.
SMTP takılırsa istek bekleyebilir.	Gönderime 30 sn zaman aşımı konuldu; hata loguna yalnızca SMTP durum kodu yazılır.
E-posta bombardımanı riski.	Hesap başına gönderim sınırı (en az 60 sn aralık, saatte en çok 5) ve IP bazlı oran sınırı.

8. Servislerin Karşılaştırmalı Özeti

Servis	Rolü	Neden seçildi	Ücretsiz plan sınırı
Render	API barındırma	Docker + Blueprint, otomatik dağıtım, yönetilen HTTPS, gizli anahtar üretimi	15 dk boşta uyur; SMTP portları kapalı
Neon	Veritabanı	Sunucusuz gerçek PostgreSQL, uyuyabilen işlem birimi, branch ile anlık kopya	Aylık işlem süresi; kısa geçmiş penceresi
Brevo	E-posta	2525 portu ile Render'dan gönderim, standart SMTP, teslim takibi	Günde 300 e-posta
GitHub Pages	Frontend	Statik site, depo ile entegre, ücretsiz	Statik içerik
GitHub Actions	CI ve yedek	Backend testleri, frontend derleme, gece yedeği	Herkese açık depoda ücretsiz

Bu yapının toplam aylık maliyeti bugün **0 TL**'dir. Tek planlanan gider, Aşama 2'deki `plantobee.com` alan adıdır (yılda yaklaşık 10–11 $).

9. Dağıtım, Test ve Yedekleme

Süreç	Nasıl çalışıyor
Backend yayını	`main`'e push → Render Docker imajını derler → sağlık kontrolü geçince yeni sürüm yayında → migration'lar açılışta uygulanır.
Frontend yayını	`main`'e push → GitHub Actions `frontend`'i derler → GitHub Pages'e yayınlar (`/PlanToBee/app/`).
Backend testleri	Backend'e dokunan her PR'da: derleme, bekleyen model değişikliği kontrolü, Web Push öz testi ve uçtan uca API senaryosu (geçici PostgreSQL ile).
Gece yedeği	Her gece 03:17 (TSİ): sabit sürümlü `postgres` imajıyla `pg_dump`, gpg AES-256 ile şifreleme, 30 gün artifact olarak saklama.
Geri yükleme	Ayrı iş akışı; yalnızca boş veritabanına, tek işlem olarak yükler, tablo başına satır sayısını raporlar.
Geri dönüş	Her büyük özellikten önce git etiketi (`rollback-oncesi-...`) ve Neon branch alınır; adımlar `docs/DEPLOY.md`'de.

10. Yol Haritası: Aşama 2

Veritabanı ve API aynen kalır; yalnızca adresler değişir:

Adres	İçerik	Servis
https://plantobee.com	Site	Cloudflare Pages
https://api.plantobee.com	API	Render (özel alan adı)
noreply@plantobee.com	Gönderen adres	Brevo (SPF/DKIM doğrulamalı)

`api.plantobee.com` kalıcı adres olacağı için ileride bir mobil uygulama da aynı API'yi kullanabilir. Trafik arttığında Render'ın ücretli planına geçmek uyku gecikmesini ortadan kaldırır; Neon ve Brevo tarafında da aynı hesapla plan yükseltmek yeterlidir, kod değişikliği gerekmez.